

VITyarthi - Build Your Own Project

Smart Expense Tracker & Budget Analyzer

A multi-user, menu-driven Python application with budget alerts and spending analytics

Student Name	Yash Tripathi
Registration No.	26BCE10516
Course	Python Programming
Submission Date	30 September 2026
GitHub Repository	https://github.com/yash26bce10516/smart-expense-tracker

Table of Contents

1. Cover Page
2. Introduction
3. Problem Statement
4. Functional Requirements
5. Non-Functional Requirements
6. System Architecture
7. Design Diagrams
8. Design Decisions & Rationale
9. Implementation Details
10. Screenshots / Results
11. Testing Approach
12. Challenges Faced
13. Learnings & Key Takeaways
14. Future Enhancements
15. References

2. Introduction

Managing personal money is one of the first real-world problems a student faces. Most people track spending loosely - in memory, in a notes app or in a spreadsheet - and only realise they have overspent after the month is over. **Smart Expense Tracker & Budget Analyzer** is a Python application that makes tracking effortless and, more importantly, gives *early warnings* while there is still time to change behaviour.

The project applies the core concepts of the Python course: functions, classes and dataclasses, modules and packages, file handling (CSV), exception handling, the standard library (sqlite3, hashlib, hmac, logging, csv, calendar, unittest), and clean layered design. It is a menu-driven command-line program with SQLite storage, so it runs anywhere Python 3.9+ is installed with **no third-party dependencies**.

Key capabilities: secure multi-user accounts; expense CRUD with filtering; per-category monthly budgets with WARNING (80%) and EXCEEDED (100%) alerts; monthly reports with ASCII charts; a trend view and a linear month-end forecast; and CSV import/export.

3. Problem Statement

Students and young professionals often lose track of where their money goes. Spreadsheets are tedious to maintain; many mobile apps are cloud-based (privacy concerns), ad-supported or too complex; and very few warn the user *before* a category budget is exceeded. The result is that overspending is discovered too late.

Objectives

- Provide a fast, private, offline way to record and manage expenses.
- Let each user define monthly budgets per category and alert them at 80% and 100% usage.
- Give clear analytics (monthly summary, category breakdown, trend, forecast) to support better decisions.
- Protect user data with hashed passwords, strict per-user data isolation and validated input.
- Deliver a well-structured, tested, documented and maintainable Python code base.

Scope. In scope: accounts, expense CRUD, budgets/alerts, analytics, CSV import/export, logging, tests. Out of scope: GUI/web/mobile front-ends, multi-currency, cloud sync and bank integration (see Future Enhancements). **Target users:** college students and young professionals who want a simple private expense log.

4. Functional Requirements

The system is organised into **five major functional modules** (the guidelines require at least three), each with a clear input/output contract.

ID	Module	Requirement	Input -> Output
FR1	User management	Register a new account with a unique username and a password of at least 6 characters.	username, password -> new user / error
FR2	User management	Log in with username and password; unknown user and wrong password give the same message.	credentials -> session user / error

ID	Module	Requirement	Input -> Output
FR3	Expense management	Add an expense (amount, category, description, date).	4 fields -> saved expense id
FR4	Expense management	View expenses, filtered by month, category and/or description keyword; show total.	filters -> table + total
FR5	Expense management	Edit any field of an existing expense (Enter keeps current value).	id + new values -> updated expense
FR6	Expense management	Delete an expense after confirmation.	id + y/N -> deleted
FR7	Budgets & alerts	Set / update a monthly budget for a category (upsert).	category, limit -> budget
FR8	Budgets & alerts	Show spent vs limit per category for a month with level OK / WARNING ($\geq 80\%$) / EXCEEDED ($\geq 100\%$).	month -> status table
FR9	Budgets & alerts	Automatically warn right after adding an expense that pushes a category over a threshold.	new expense -> alert message
FR10	Analytics	Monthly report: count, total, largest, daily average, category ASCII bar chart, top-3 expenses.	month -> report
FR11	Analytics	Multi-month spending trend and linear month-end forecast.	month -> trend + projection
FR12	Import / export	Export all expenses to CSV; import from CSV, skipping and reporting invalid rows by line number.	path -> counts + error list

Workflow. Start -> Login/Register -> Main menu -> choose a feature -> input is validated -> data is read/written in a database transaction -> result (and any alert) is displayed -> back to the main menu -> Logout. The full flowchart is in Section 7.

5. Non-Functional Requirements

The guidelines require at least four; the project specifies and implements eight. Each is verifiable.

ID	Category	Requirement	How it is met / verified
NFR1	Security	Passwords never stored in plain text; users cannot access each other's data; no SQL injection.	Salted PBKDF2-HMAC-SHA256 (600,000 iterations), constant-time comparison; every query filters by user_id; parameterised SQL. Tests: hashing, isolation, injection.
NFR2	Performance	Typical operations complete in well under one second with thousands of records.	Index on (user_id, date); SQL aggregation. Measured with 10,000 rows: list month 7 ms, three analytics queries 15 ms, single add 5 ms.
NFR3	Usability	Clear menus, sensible defaults (today's date / current month), friendly error messages.	Prompts show defaults; errors never show stack traces; the password prompt hides input.
NFR4	Reliability	A failed operation must never corrupt data or crash the session.	Transactional Database.session() with rollback; DB CHECK constraints; last-resort exception guard in the CLI loop.
NFR5	Maintainability	Layered, modular code with single-responsibility modules.	12 modules (presentation / business / data / cross-cutting); docstrings; constants in config.py.

ID	Category	Requirement	How it is met / verified
NFR6	Error-handling strategy	Every failure maps to a specific, user-readable message.	Custom hierarchy <code>TrackerError</code> -> <code>ValidationError</code> / <code>AuthError</code> / <code>NotFoundError</code> , caught in one dispatcher.
NFR7	Logging / monitoring	Important events and failures are recorded for diagnosis.	Rotating log file (200 KB x 2 backups) with timestamp, level and module; failed logins logged as warnings.
NFR8	Resource efficiency / portability	Small footprint; runs anywhere without installation steps.	Standard library only; single-file SQLite database; connections are opened per operation and always closed.

6. System Architecture

The application follows a **layered architecture** so each concern can be changed or tested independently:

- **Presentation layer** - `cli.py` and `main.py`: menus, prompts and output only; no business rules.
- **Business-logic layer** - `auth.py`, `expense_service.py`, `budget_service.py`, `analytics.py`, `exporter.py`: all rules and calculations.
- **Data layer** - `database.py`: schema creation and transactional sessions over a SQLite file.
- **Cross-cutting** - `validators.py`, `exceptions.py`, `logger_setup.py`, `config.py`, `models.py` used by all layers.

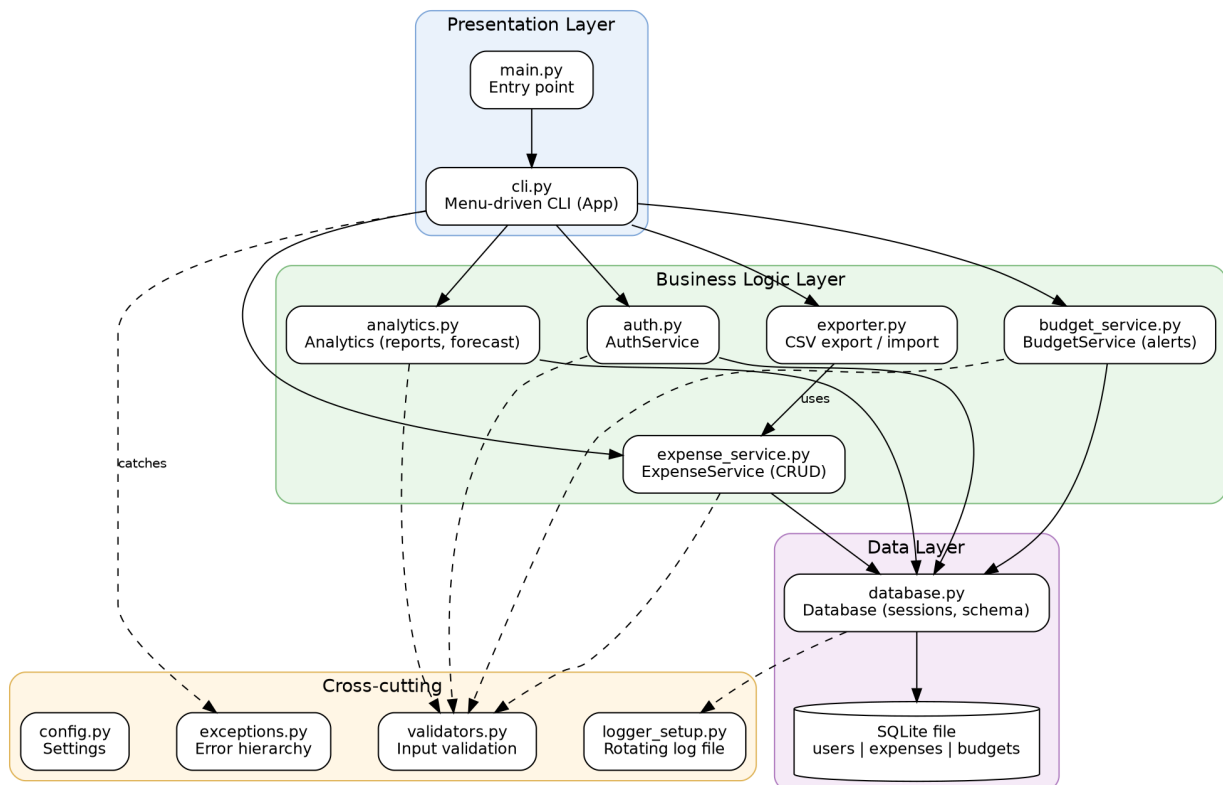


Figure 1 - System architecture diagram

Dependencies point downwards only (CLI -> services -> database), and the CLI receives its input/output functions by injection, which is what allows the complete menu workflow to be tested automatically without a keyboard.

7. Design Diagrams

7.1 Use Case Diagram

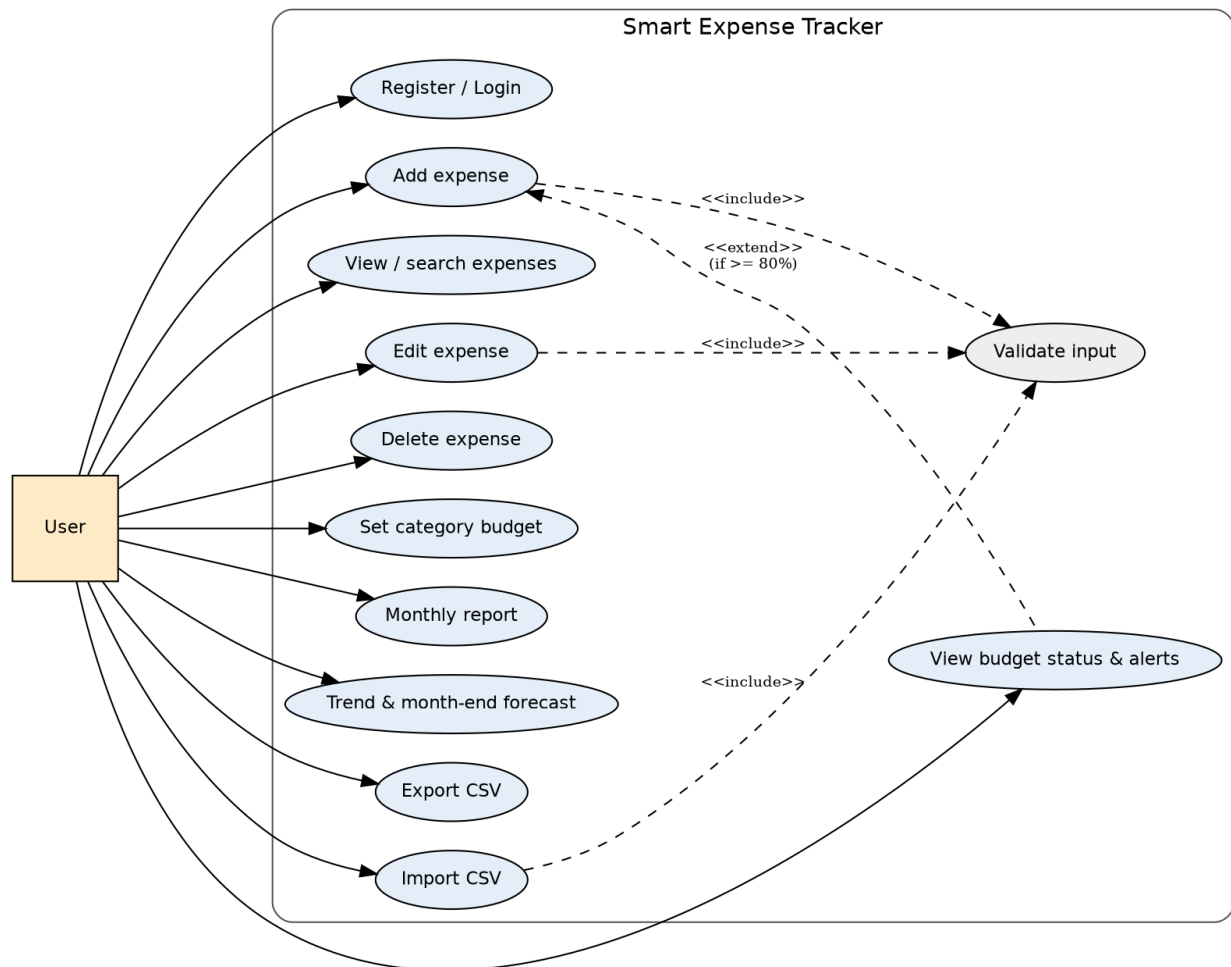


Figure 2 - Use case diagram (single actor: User)

Adding an expense *includes* input validation and *extends* to a budget alert when the 80% threshold is crossed.

7.2 Workflow Diagram

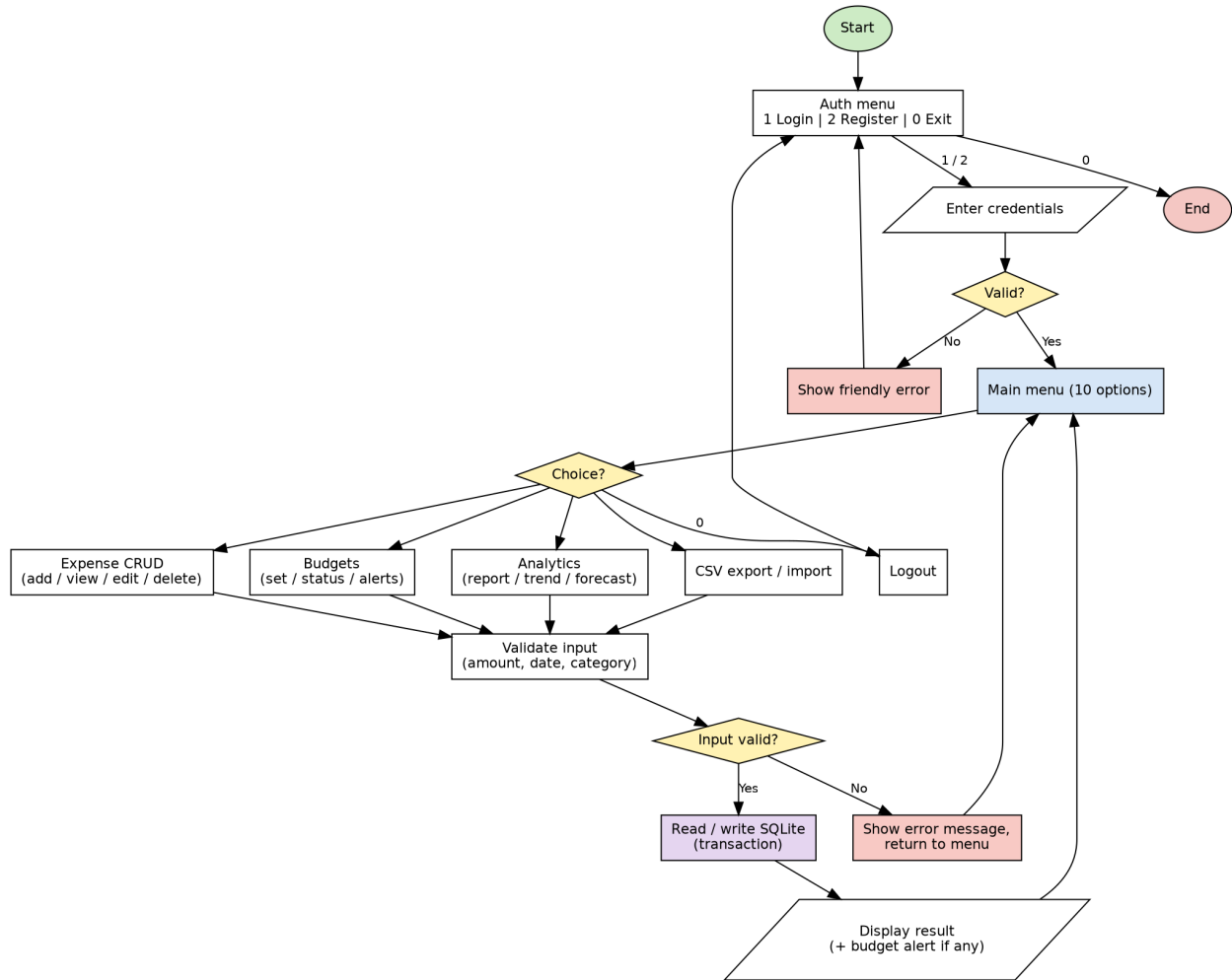


Figure 3 - Process / workflow diagram

7.3 Sequence Diagram

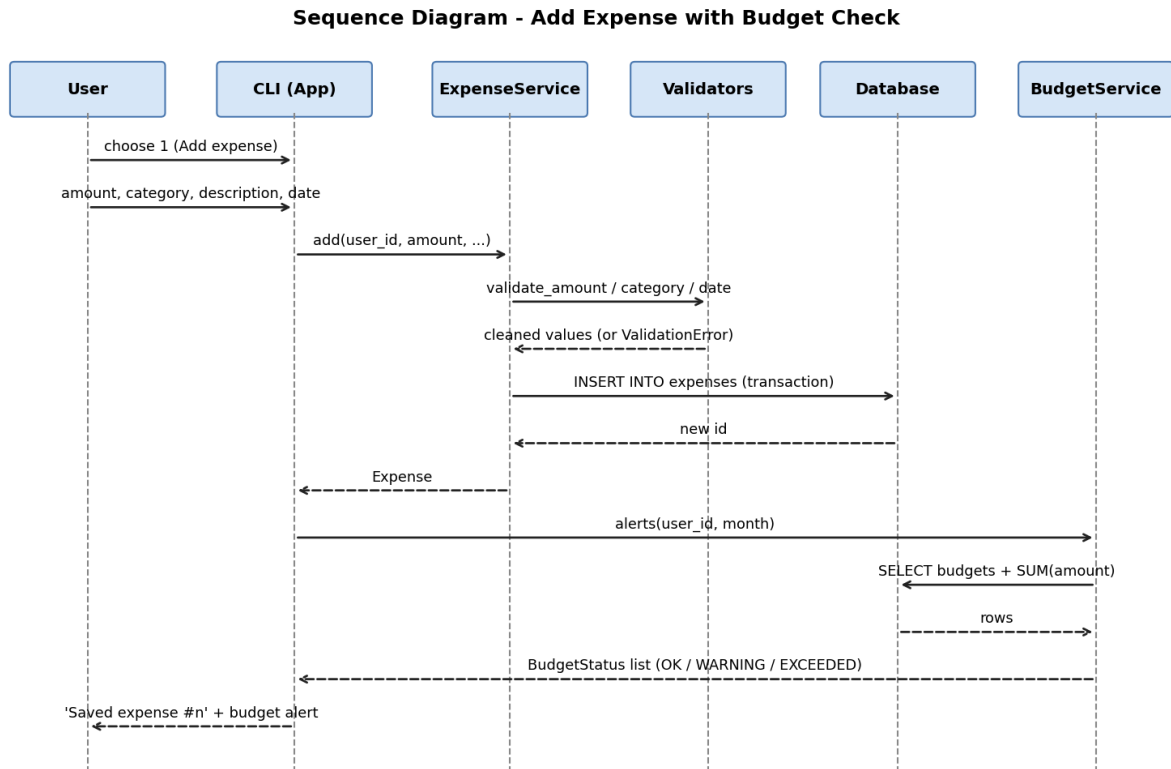


Figure 4 - Sequence diagram: adding an expense with budget check

7.4 Class Diagram

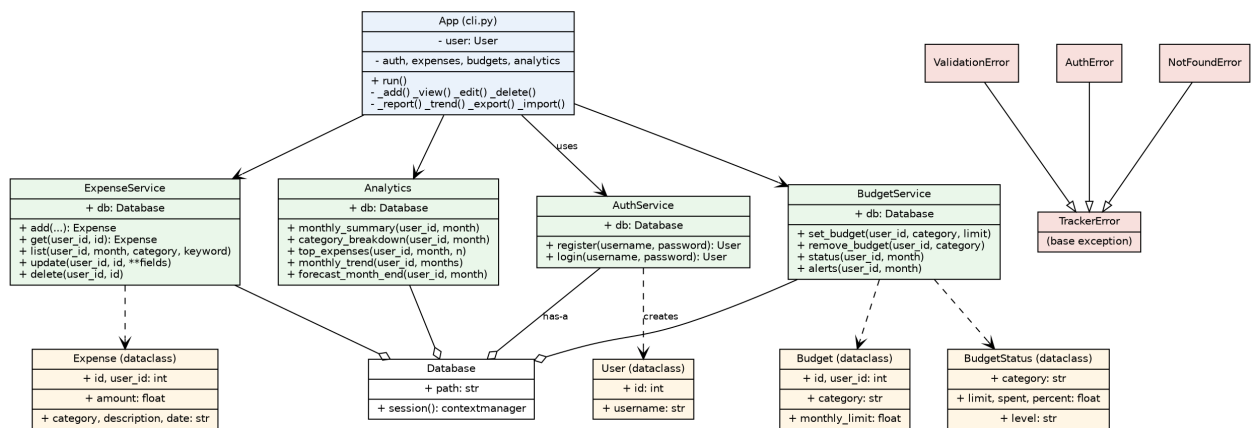


Figure 5 - Class diagram

7.5 ER Diagram and Schema

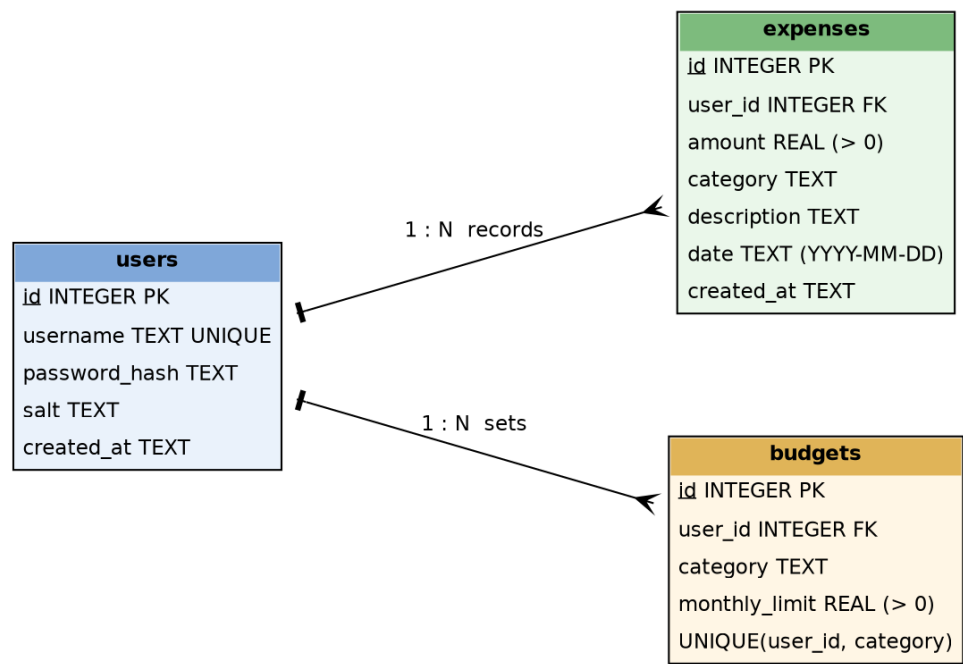


Figure 6 - Entity-relationship diagram

Table	Columns	Constraints
users	id, username, password_hash, salt, created_at	PK id; username UNIQUE
expenses	id, user_id, amount, category, description, date, created_at	PK id; FK user_id -> users ON DELETE CASCADE; CHECK amount > 0; index (user_id, date)
budgets	id, user_id, category, monthly_limit	PK id; FK user_id; CHECK monthly_limit > 0; UNIQUE (user_id, category)

8. Design Decisions & Rationale

Decision	Alternatives considered	Rationale
SQLite for storage	CSV / JSON files; MySQL	Zero setup, part of the standard library, ACID transactions, supports SQL aggregation for reports, and a single-file database is easy to back up.
Layered modules with service classes	One large script	Separates UI from rules and data; each service is unit-testable; meets the modular-code expectation.
PBKDF2 with per-user random salt (stdlib)	Plain text; unsalted SHA-256; bcrypt (needs pip)	Plain/unsalted hashes are unsafe; PBKDF2 is available without dependencies and is a recognised standard. Iterations are configurable.
Identical error for bad user / bad password	Distinct messages	Prevents attackers from discovering which usernames exist (user enumeration).
All queries scoped by user_id	Trusting the UI to pass the right id	Enforces isolation at the data layer; a user cannot read, edit or delete another user's expense (tested).
Dates stored as ISO text (YYYY-MM-DD)	Unix timestamps	Sorts correctly as text, human-readable, and month filtering is a simple substr(date,1,7).
Amounts rounded to 2 decimals, limited to 10,000,000	Decimal type	Adequate for a personal tracker and simple; the limit blocks absurd or accidental values. (Decimal is listed as a future improvement.)
Dependency-injected input/output in the CLI	Direct input()/print()	Lets the entire menu workflow be tested end-to-end with scripted input.
Custom exception hierarchy	Generic Exception / return codes	One dispatcher converts expected errors to friendly messages while unexpected ones are logged.
Linear forecast (spent / days elapsed x days in month)	Regression / ML	Transparent, explainable and correct for the data available; past months return the real total and future months return 0.

9. Implementation Details

The code base is a Python package of 12 modules plus an entry point and 7 test files. Line counts (non-blank) are measured from the repository:

Module	Responsibility	Lines
cli.py	Menu-driven interface; input/output injected	168
auth.py	Register / login, PBKDF2 hashing	47
expense_service.py	Expense CRUD and filtered listing	77
budget_service.py	Budgets, status, alert levels	56
analytics.py	Summary, breakdown, trend, forecast, ASCII chart	75
exporter.py	CSV export / import with row-level errors	43
database.py	Schema + transactional session context manager	56
validators.py	Validation of every input type	47
models.py	Dataclasses: User, Expense, Budget, BudgetStatus	27

Module	Responsibility	Lines
exceptions.py	Custom exception hierarchy	9
logger_setup.py	Rotating file logger	20
config.py	Constants and environment overrides	15
Total (application)		640
tests/ (7 test files + fixtures)	Unit and end-to-end tests	311

Key implementation techniques

1. Transactional database sessions - a context manager guarantees commit on success, rollback on error and closing of the connection:

```
@contextmanager
def session(self):
    conn = sqlite3.connect(self.path); conn.row_factory = sqlite3.Row
    conn.execute("PRAGMA foreign_keys = ON")
    try:
        yield conn; conn.commit()
    except Exception:
        conn.rollback(); log.exception("Transaction rolled back"); raise
    finally:
        conn.close()
```

2. Secure password storage - a random 16-byte salt per user and a constant-time comparison:

```
def _hash_password(password, salt):
    return hashlib.pbkdf2_hmac("sha256", password.encode(), salt,
                               config.PBKDF2_ITERATIONS).hex()

...
if not hmac.compare_digest(expected, actual):
    raise AuthError("Invalid username or password.")
```

3. Budget classification and forecast - the alert level is a pure function (easy to test), and the forecast handles past, current and future months:

```
def classify(percent):
    if percent >= 100: return "EXCEEDED"
    if percent >= config.WARNING_THRESHOLD: return "WARNING"    # 80.0
    return "OK"

projected = round(spent / elapsed * days_in_month, 2) if elapsed else 0.0
```

4. Validation at the service boundary - every service calls the validators before touching the database, so bad data cannot enter regardless of which front-end is used. **5. Safe updates** - `update()` whitelists column names through a dictionary of cleaner functions, so the dynamic SET clause can never contain user-supplied SQL.

Version control. The project was developed in Git with a series of small, descriptive commits (project skeleton, data layer, authentication, expense CRUD, budgets, analytics, CSV, CLI, tests, documentation).

10. Screenshots / Results

The screenshots below are the actual program output from a scripted demonstration session (user *student01*, September 2026 data).

```
* Adding expenses - budget alerts

Choose: 1
Categories: Food, Transport, Rent, Utilities, Shopping, Health, Education, Entertainment, Other
Amount: 900
Category: Entertainment
Description: Movie and dinner
Date [2026-09-30]: 2026-09-12
Saved expense #4.
!! Budget WARNING: Entertainment is at 90.0% (Rs. 900.00 of Rs. 1,000.00)
```

Figure 7 - Adding expenses: WARNING at 90% of Entertainment budget

```
* View / filter expenses (month = 2026-09)

Filter month YYYY-MM (blank = all): 2026-09
Filter category (blank = all):
Search description (blank = none):

  ID  Date       Category      Amount  Description
  --  -
  7   2026-09-22 Education    1,250.00 Python reference book
  6   2026-09-20 Food         1,500.00 Party snacks
  5   2026-09-14 Food         1,800.00 Restaurant with friends
  4   2026-09-12 Entertainment  900.00 Movie and dinner
  2   2026-09-04 Transport     350.00 Metro card recharge
  1   2026-09-03 Food         1,200.00 Weekly groceries
  3   2026-09-01 Rent         8,500.00 Hostel rent

Total: Rs. 15,500.00 (7 items)
```

Figure 8 - Viewing expenses filtered by month

```
* Budget status & alerts

Month (YYYY-MM) [2026-09]: 2026-09

Entertainment    Rs. 900.00 / Rs. 1,000.00    90.0% [WARNING]
Food             Rs. 4,500.00 / Rs. 4,000.00  112.5% [EXCEEDED]
Transport        Rs. 350.00 / Rs. 1,500.00    23.3% [OK]
```

Figure 9 - Budget status: Food is EXCEEDED (112.5%)

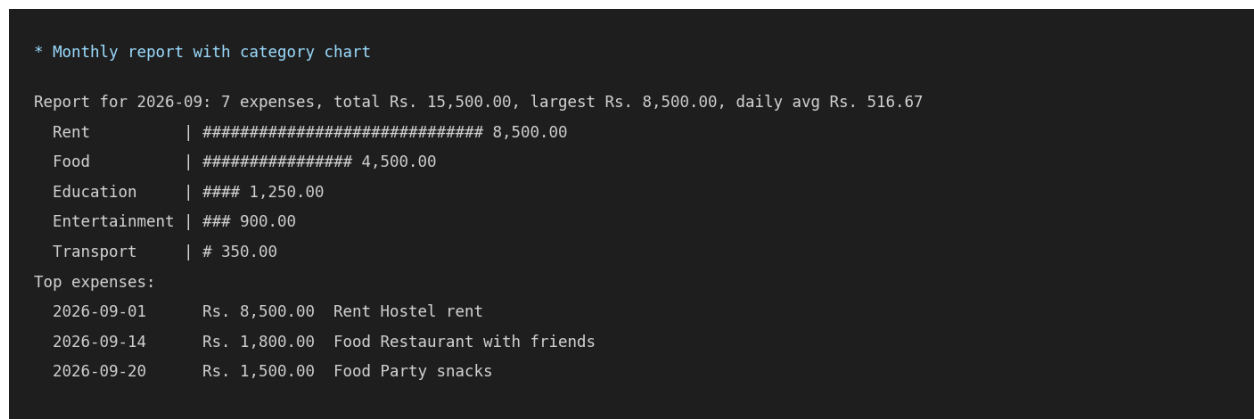


Figure 10 - Monthly report with category bar chart and top expenses

Result summary. For the demo data the report correctly computes 7 expenses totalling Rs. 15,500.00 (daily average Rs. 516.67); Food spend of Rs. 4,500 against a Rs. 4,000 budget is flagged EXCEEDED at 112.5%; Entertainment at 90% is flagged WARNING; and the trend view lists August (Rs. 1,400) before September (Rs. 15,500).

11. Testing Approach

Testing uses Python's built-in `unittest`. Each test gets a fresh temporary SQLite database, so tests are independent and never touch real data. Password hashing uses a low iteration count during tests only (via an environment variable) to keep the suite fast.

Test file	What is verified	Tests
test_validators.py	Valid / invalid amounts, categories, dates, months, usernames, passwords, description length	6
test_auth.py	Login success, identical errors for bad user/password, duplicate username, weak password, no plain-text password	5
test_expense_service.py	CRUD, filters, partial update, delete, user isolation, SQL-injection text stored safely	7
test_budget_service.py	Threshold boundaries (79.9 / 80 / 100), upsert, monthly scoping, alerts, remove	5
test_analytics.py	Summary, empty month, breakdown, top-N, trend order, forecast (mid / past / future), ASCII chart	9
test_exporter.py	CSV round-trip incl. commas, bad rows skipped with line numbers, missing file, bad header	3
test_cli.py	End-to-end scripted sessions: full workflow and friendly error handling	2

Result: 37 tests, all passing (`python -m unittest discover -v`). **Validation / performance test:** with 10,000 expense rows, listing one month (1,073 rows) took 7 ms, three analytics queries took 15 ms and a single insert took 5 ms - comfortably meeting NFR2. **Manual testing:** the interactive program was exercised with invalid menu options, non-numeric ids, negative amounts, malformed dates and an unknown login.

12. Challenges Faced

- **Testing an interactive program.** Menus that call `input()` are hard to test. Solved by injecting the input, output and password functions into the App class so a full session can be scripted.
- **Keeping users' data separate.** Forgetting a single `user_id` filter would leak data. Solved by making every service method take the user id and adding explicit isolation tests.
- **Month-end forecasting edge cases.** A naive formula gives wrong answers for past and future months. Solved by treating a finished month as fully elapsed and a future month as zero, each covered by a test.
- **A bug found by the demo run.** The trend chart initially sorted months by amount instead of chronologically. It was found while reviewing real output, fixed by adding a `sort_by_value` option, and locked in with a regression test.
- **Password-hash strength vs. test speed.** Strong hashing (600,000 iterations, about 0.6 s) makes tests slow. Solved with a configurable iteration count that tests lower via an environment variable.
- **Money and floating point.** Floats can produce values like 0.1+0.2. Amounts are rounded to two decimals at the validation boundary; using Decimal is noted as a future improvement.

13. Learnings & Key Takeaways

- Separating presentation, business logic and data makes code easier to test, change and explain.
- Validating at the service boundary is more robust than trusting the user interface.
- Parameterised SQL, salted hashing and least-privilege data access are practical, low-cost security habits.
- Context managers, dataclasses, custom exceptions and logging make Python code clearer and safer.
- Writing tests early exposes design problems (such as untestable `input()`) and edge cases before users find them.
- Reviewing real program output, not just passing tests, finds bugs that tests did not anticipate.

14. Future Enhancements

- A graphical (Tkinter) or web (Flask) front-end with real charts using Matplotlib.
- Recurring expenses (rent, subscriptions) and reminders.
- Use of the Decimal type or integer paise for exact currency arithmetic; multi-currency support.
- Custom categories, tags and receipt attachments.
- Machine-learning based category suggestion and anomaly detection for unusual spends.
- Cloud backup / sync and bank or UPI statement import; account lockout after repeated failed logins.

15. References

[1] Python Software Foundation. *Python 3 Documentation - sqlite3, hashlib, hmac, logging, csv, unittest, dataclasses*. <https://docs.python.org/3/>

[2] OWASP. *Password Storage Cheat Sheet*. https://cheatsheetseries.owasp.org/cheatsheets/Password_Storage_Cheat_Sheet.html

- [3] NIST. *SP 800-132: Recommendation for Password-Based Key Derivation (PBKDF2)*. <https://csrc.nist.gov/publications/detail/sp/800-132/final>
- [4] SQLite Consortium. *SQLite Documentation*. <https://www.sqlite.org/docs.html>
- [5] Graphviz. *DOT Language documentation* (used to generate the diagrams). <https://graphviz.org/doc/info/lang.html>
- [6] VITyarthi. *Build Your Own Project - General Project Instructions & Submission Guidelines*.